Part 8

Terraform with Jenkins CI/CD

Professional Interview Notes · Beginner to Advanced · English Technical Notes with Simple Kannada Explanations

These notes explain how to automate Terraform infrastructure deployments using Jenkins, including pipeline automation, AWS credentials, plan and apply stages, manual approvals, security, state management, and production troubleshooting.

Each topic follows your requested five headings:

1. Definition — What is it?
2. Purpose — Why do we use it?
3. Working — How does it work?
4. Practical Example — How do we use it in a project?
5. Scenario-based Interview Questions — How can it be asked in interviews?

Topics covered

1. Terraform and Jenkins CI/CD Fundamentals
2. Jenkins Pipeline and Jenkinsfile
3. GitHub Integration and Webhooks
4. Terraform Initialization and Validation
5. AWS Credentials and Secure Authentication
6. Terraform Plan and Apply Automation
7. Manual Approval Before Production Deployment
8. Remote State and State Locking
9. Pipeline Parameters and Environment Management
10. Terraform CI/CD Security Best Practices
11. Terraform Destroy and Approval Controls
12. Pipeline Failure Troubleshooting
13. End-to-End AWS Terraform Jenkins Pipeline
14. Real-Time Production Scenarios and Interview Q&A

1. Terraform and Jenkins CI/CD Fundamentals

1.1 Definition — What is Terraform with Jenkins CI/CD?

Terraform is an Infrastructure as Code (IaC) tool used to provision and manage infrastructure. Jenkins is an automation server used to execute pipelines and automate software delivery and infrastructure workflows.

When integrated, Jenkins runs Terraform commands automatically to validate infrastructure code, generate plans, and deploy approved changes to cloud platforms such as AWS.

Kannada explanation: Terraform ಬಳಸಿ AWS resources create ಮಾಡುತ್ತೇವೆ. Jenkins ಆ Terraform commands ಅನ್ನು automatic ಆಗಿ run ಮಾಡುತ್ತದೆ. ಉದಾಹರಣೆಗೆ GitHubನಲ್ಲಿ code push ಮಾಡಿದಾಗ Jenkins pipeline start ಆಗಿ Terraform plan ತಯಾರಿಸಬಹುದು.

1.2 Purpose — Why do we use it?

- Automate infrastructure deployment.
- Reduce manual errors and repetitive commands.
- Standardize infrastructure changes.
- Integrate Terraform with Git-based code review.
- Add approval gates before production changes.
- Maintain logs and audit trails of deployments.
- Support repeatable deployments across environments.

Kannada explanation: ಪ್ರತಿಬಾರಿಯೂ terminalನಲ್ಲಿ Terraform commands manually run ಮಾಡುವ ಬದಲು Jenkins ಮೂಲಕ automate ಮಾಡಬಹುದು. ಇದರಿಂದ ಸಮಯ ಉಳಿಯುತ್ತದೆ, errors ಕಡಿಮೆಯಾಗುತ್ತವೆ ಮತ್ತು ಯಾರು ಯಾವ change deploy ಮಾಡಿದರು ಎಂಬ history ಸಿಗುತ್ತದೆ.

1.3 Working — How does it work?

```
1. Developer commits code
   Terraform .tf files -> GitHub
        |
        v
2. Jenkins pipeline starts
   Webhook or manual build
        |
        v
3. Validate and plan
   fmt -> init -> validate -> plan
        |
        v
4. Approval gate
   Review planned infrastructure changes
        |
        v
5. Apply approved changes
   Terraform -> AWS resources
```

The exact pipeline may vary by project. For production, a successful validation or plan must not automatically imply permission to apply; approval and access controls should be designed separately.

Kannada explanation: Developer code push ಮಾಡುತ್ತಾರೆ. Jenkins code checkout ಮಾಡಿ validate ಮತ್ತು plan run ಮಾಡುತ್ತದೆ. Plan review ಆದ ನಂತರ ಅನುಮತಿ ಸಿಕ್ಕರೆ ಮಾತ್ರ apply ಮಾಡಿ AWSನಲ್ಲಿ changes ಮಾಡುತ್ತದೆ.

1.4 Practical Example — AWS Terraform project

Suppose your Terraform repository creates:

- An AWS VPC and subnets.
- Security groups.
- EC2 instances.

- An EKS cluster and node group.
- IAM roles and policies.

A Jenkins pipeline can automate the workflow:

```
GitHub Push
    ↓
Jenkins Checkout
    ↓
Terraform Init
    ↓
Terraform Format Check
    ↓
Terraform Validate
    ↓
Terraform Plan
    ↓
Manual Approval
    ↓
Terraform Apply
```

For your existing AWS Terraform project, the same process can be used to automate infrastructure changes. Keep the backend, AWS credentials, and Terraform working directory configured correctly.

1.5 Scenario-based Interview Questions

Q1. Why do you integrate Terraform with Jenkins?

Answer: Jenkins automates Terraform execution, integrates with source control, provides consistent pipeline stages, supports approvals, and records deployment results.

Q2. A developer pushes a Terraform change to GitHub. What happens next?

Answer: A webhook or configured trigger starts Jenkins. Jenkins checks out the configured branch, initializes Terraform, validates the code, creates a plan, and proceeds to apply only according to the pipeline's approval and deployment rules.

Q3. Is Jenkins mandatory to use Terraform?

Answer: No. Terraform can run locally or through other CI/CD platforms. Jenkins is one option for automating and controlling the workflow.

Q4. What is the most important production safeguard?

Answer: Review the plan, restrict deployment permissions, protect state and credentials, and require appropriate approval before applying risky production changes.

2. Jenkins Pipeline and Jenkinsfile

2.1 Definition — What is a Jenkins Pipeline?

A Jenkins Pipeline is an automated sequence of stages and steps that Jenkins executes to perform a workflow.

A Jenkinsfile is a text file that defines the pipeline. It is commonly stored in the same Git repository as the Terraform code, allowing pipeline changes to be reviewed and version-controlled.

Two common pipeline styles are:

- Declarative Pipeline: Uses a structured syntax with pipeline, agent, stages, and steps.

- Scripted Pipeline: Uses Groovy-based scripting for more flexible control flow.

> **Kannada explanation:** Jenkins Pipeline ಅಂದ್ರೆ ಕೆಲಸಗಳನ್ನು ಒಂದರ ನಂತರ ಒಂದು automatic ಆಗಿ ಮಾಡಿಸುವ workflow. Jenkinsfileನಲ್ಲಿ ಯಾವ stageನಲ್ಲಿ ಯಾವ command run ಆಗಬೇಕು ಅಂತ ಬರೆಯುತ್ತೇವೆ.

2.2 Purpose — Why do we use it?

- Define repeatable automation.
- Keep pipeline code in Git.
- Separate checkout, validation, plan, approval, and apply stages.
- Simplify troubleshooting through stage-specific logs.
- Support reusable pipeline standards across projects.

> **Kannada explanation:** Jenkinsfileನಲ್ಲಿ ಎಲ್ಲಾ stages ಅನ್ನು ಕ್ರಮವಾಗಿ ಬರೆಯಬಹುದು. ಒಂದು stage fail ಆದರೆ ಯಾವ ಹಂತದಲ್ಲಿ ಸಮಸ್ಯೆ ಆಯಿತು ಎಂದು logs ಮೂಲಕ ಸುಲಭವಾಗಿ ತಿಳಿಯಬಹುದು.

2.3 Working — How does it work?

A declarative pipeline usually contains:

- pipeline: The main pipeline definition.
- agent: Where the pipeline runs.
- environment: Environment variables.
- parameters: User-provided build inputs.
- stages: Major workflow stages.
- steps: Commands and actions inside each stage.
- post: Actions performed after the pipeline or stage finishes.

2.4 Practical Example — Basic Terraform Jenkinsfile

```
pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Terraform Init') {
            steps {
                bat 'terraform init -input=false'
            }
        }

        stage('Format Check') {
            steps {
                bat 'terraform fmt -check -recursive'
            }
        }

        stage('Validate') {
            steps {
                bat 'terraform validate'
            }
        }

        stage('Plan') {
            steps {
                bat 'terraform plan -input=false'
            }
        }
    }
}
```

This example uses bat because it is intended for a Windows Jenkins agent. For Linux agents, use sh instead.

The example assumes Terraform is installed, the workspace contains the correct configuration, the backend is accessible, and any required credentials are configured.

> **Kannada explanation:** Windows Jenkins agentನಲ್ಲಿ bat command ಬಳಸುತ್ತೇವೆ. Linux agentನಲ್ಲಿ sh ಬಳಸಬೇಕು. ಈ pipeline Git code checkout ಮಾಡಿ Terraform init, format check, validate ಮತ್ತು plan run ಮಾಡುತ್ತದೆ.

2.5 Scenario-based Interview Questions

Q1. What is the difference between a Jenkinsfile and a Jenkins job?

Answer: A Jenkins job is the configured automation task in Jenkins. A Jenkinsfile is the pipeline definition, usually stored in source control, that tells Jenkins how to execute the workflow.

Q2. Your Jenkinsfile works on Linux but fails on Windows. What could be the reason?

Answer: The pipeline may use shell commands or Unix-specific paths that do not work on Windows. Use the appropriate sh or bat step and verify executable paths and environment variables.

Q3. Why should a Jenkinsfile be stored in Git?

Answer: It enables version control, peer review, history tracking, and consistent pipeline configuration across builds.

3. GitHub Integration and Webhooks

3.1 Definition — What is GitHub integration in Jenkins?

GitHub integration connects a Jenkins pipeline to a GitHub repository so that Jenkins can retrieve the source code and optionally start builds when repository events occur.

A webhook is an HTTP notification sent by GitHub to a configured endpoint when an event, such as a push, occurs.

> **Kannada explanation:** Jenkinsಗೆ GitHub repository connect ಮಾಡಿದರೆ Jenkins code ಅನ್ನು checkout ಮಾಡಬಹುದು. Webhook enable ಮಾಡಿದರೆ developer code push ಮಾಡಿದಾಗ Jenkinsಗೆ notification ಹೋಗಿ build start ಆಗಬಹುದು.

3.2 Purpose — Why do we use it?

- Automatically retrieve the latest committed Terraform code.
- Start pipelines when relevant code changes occur.
- Keep infrastructure changes version-controlled.
- Reduce manual build triggering.
- Support continuous integration.

> **Kannada explanation:** Developer ಪ್ರತಿಬಾರಿ Jenkinsಗೆ ಹೋಗಿ build click ಮಾಡುವ ಅಗತ್ಯ ಕಡಿಮೆಯಾಗುತ್ತದೆ. GitHubನಲ್ಲಿ code push ಮಾಡಿದಾಗ configured webhook ಮೂಲಕ Jenkins pipeline trigger ಆಗಬಹುದು.

3.3 Working — How does it work?

1. Store Terraform configuration and Jenkinsfile in GitHub.
2. Configure the Jenkins job to use the correct repository URL.
3. Select the correct branch and Jenkinsfile path.
4. Configure GitHub authentication if the repository requires it.
5. Configure a webhook or an alternative polling/trigger mechanism.
6. Push a commit and verify the Jenkins build log.

The Jenkins endpoint must be reachable from GitHub for a direct webhook to work. A Jenkins server running only on localhost is not directly reachable by GitHub's public webhook service.

3.4 Practical Example — Multibranch Pipeline

A multibranch pipeline can discover branches containing a Jenkinsfile and create branch-specific jobs.

Example repository structure:

```
terraform-project/
├── Jenkinsfile
├── main.tf
├── variables.tf
├── outputs.tf
└── modules/
    └── ec2/
```

For a repository with a feature/terraform-ci branch:

1. Commit the Terraform code and Jenkinsfile.
2. Push the branch to GitHub.

3. Configure Jenkins to discover the branch.
4. Confirm the correct Jenkinsfile is loaded.
5. Run the pipeline and inspect its checkout and plan stages.

> **Kannada explanation:** Branchನಲ್ಲಿ Jenkinsfile ಇದ್ದರೆ multibranch pipeline ಆ branchಗೆ job create ಮಾಡಬಹುದು. ಆದರೆ Jenkinsfile ಸರಿಯಾದ branchನಲ್ಲಿ ಇದೆಯೇ ಮತ್ತು repository credentials ಸರಿಯಾಗಿವೆಯೇ ಎಂದು check ಮಾಡಬೇಕು.

3.5 Scenario-based Interview Questions

Q1. You push code to GitHub, but Jenkins does not start a build. How do you troubleshoot?

Answer: Work through these checks:

1. Confirm the push reached the expected branch.
2. Check the repository URL and branch configuration.
3. Confirm the Jenkinsfile exists at the configured path.
4. Inspect webhook delivery status in GitHub.
5. Check whether Jenkins is reachable from GitHub.
6. Review Jenkins trigger configuration and logs.

Q2. Jenkins reports that it cannot find the Jenkinsfile. What could be wrong?

Answer: The file may be missing from the selected branch, stored in a different directory, named incorrectly, or referenced by the wrong script path.

Q3. How do you secure a private GitHub repository connection?

Answer: Use an approved Git credential or fine-grained token with minimum required permissions, store it in Jenkins credentials, and avoid putting tokens directly in the Jenkinsfile or repository URL.

4. Terraform Initialization and Validation

4.1 Definition — What are Terraform initialization and validation stages?

terraform init initializes the Terraform working directory, installs required providers and modules, and configures the backend.

terraform validate checks whether the Terraform configuration is syntactically valid and internally consistent.

terraform fmt -check checks whether files follow Terraform's canonical formatting.

> **Kannada explanation:** init Terraform projectಗೆ ಬೇಕಾದ providers ಮತ್ತು modules prepare ಮಾಡುತ್ತದೆ. validate codeನಲ್ಲಿ configuration errors ಇದೆಯೇ ಎಂದು check ಮಾಡುತ್ತದೆ. fmt -check code formatting ಸರಿಯಾಗಿದೆಯೇ ಎಂದು ನೋಡುತ್ತದೆ.

4.2 Purpose — Why do we use them?

- Download required providers and modules.
- Initialize remote state backends.
- Detect configuration mistakes early.
- Maintain consistent formatting.
- Fail the pipeline before infrastructure changes are attempted.

4.3 Working — How do they work?

A typical pipeline executes:

```
terraform init -input=false
terraform fmt -check -recursive
terraform validate
terraform plan -input=false
```

In Windows Jenkins pipelines, the commands can be run with bat. In Linux agents, use sh.

Important: terraform validate alone does not prove that AWS credentials are valid, that a resource can be created, or that a deployment is safe. The plan stage checks the proposed changes against the configured environment and state.

4.4 Practical Example — Jenkins validation stage

```
stage('Terraform Validation') {
    steps {
        bat 'terraform init -input=false'
        bat 'terraform fmt -check -recursive'
        bat 'terraform validate'
    }
}
```

If the configuration is in a subdirectory, select the correct working directory. A pipeline can also use a Jenkins dir('EKS-TF') block or an appropriate directory setting.

> **Kannada explanation:** Terraform files EKS-TF folderನಲ್ಲಿ ಇದ್ದರೆ pipeline ಅದೇ folderನಲ್ಲಿ commands run ಮಾಡಬೇಕು. Wrong folderನಲ್ಲಿ run ಮಾಡಿದರೆ Terraform configuration ಸಿಗದ error ಬರಬಹುದು.

4.5 Scenario-based Interview Questions

Q1. terraform init succeeds, but terraform validate fails. What do you check?

Answer: Inspect the validation error for unsupported arguments, undeclared variables, invalid references, type mismatches, or configuration syntax errors. Correct the code and rerun validation.

Q2. Why should terraform init be included in CI/CD?

Answer: Jenkins agents may start with clean workspaces. Initialization downloads the dependencies and configures the backend needed for the current run.

Q3. Does successful validation guarantee a successful apply?

Answer: No. Apply can still fail because of invalid credentials, insufficient IAM permissions, quotas, unavailable services, conflicting resources, or runtime API errors.

5. AWS Credentials and Secure Authentication

5.1 Definition — What is AWS authentication in Jenkins?

AWS authentication allows Jenkins to make authorized requests to AWS services when executing Terraform.

Common approaches include:

- Jenkins credentials containing AWS access keys and secret keys.

- IAM roles attached to AWS-hosted Jenkins agents.
- Short-lived credentials obtained through an approved identity federation mechanism.

For AWS-hosted workloads, IAM roles and temporary credentials are generally preferable to long-lived access keys when feasible.

> **Kannada explanation:** Terraformಗೆ AWS resources create ಮಾಡಲು AWS permissions ಬೇಕು. Jenkinsಗೆ ಆ permissions ಅನ್ನು ಸುರಕ್ಷಿತವಾಗಿ ಕೊಡಬೇಕು. ಸಾಧ್ಯವಾದರೆ permanent access keys ಬದಲು IAM role ಮತ್ತು temporary credentials ಬಳಸುವುದು ಉತ್ತಮ.

5.2 Purpose — Why do we use it?

- Authenticate Terraform operations to AWS.
- Grant only the permissions needed for the job.
- Avoid hardcoding secrets in source code.
- Separate development and production access.
- Reduce the risk of credential leaks.
- Support credential rotation and auditing.

5.3 Working — How does it work?

A typical authentication flow is:

1. Jenkins obtains credentials from an approved credentials provider.
2. The AWS CLI or Terraform AWS provider uses those credentials.
3. AWS verifies the identity and checks IAM permissions.
4. Terraform performs only the operations permitted by that identity.

When using temporary credentials, the session may also require a session token.

Important distinction

Authentication answers: “Who are you?”

Authorization answers: “What are you allowed to do?”

A successful aws sts get-caller-identity proves that AWS recognizes the current identity. It does not prove that the identity has all permissions needed for Terraform.

5.4 Practical Example — Jenkins credentials binding

For an existing Windows Jenkins agent, an illustrative pipeline stage is:

```
stage('Verify AWS Identity') {
    steps {
        withCredentials([
            [
                $class: 'AmazonWebServicesCredentialsBinding',
                credentialsId: 'aws-jenkins-credentials'
            ]
        ]) {
            bat 'aws sts get-caller-identity'
        }
    }
}
```

This example requires a compatible AWS Credentials plugin and credentials entry in Jenkins. The exact binding syntax may differ with the plugin version and configuration.

For production, prefer an approved role-based authentication design when available. Never print secret environment variables or use set/env commands that dump credentials into logs.

> **Kannada explanation:** credentialsId ಮೂಲಕ Jenkinsನಲ್ಲಿ save ಮಾಡಿದ credentials ಬಳಸುತ್ತೇವೆ. Secret ಅನ್ನು Jenkinsfileನಲ್ಲಿ ನೇರವಾಗಿ ಬರೆಯುವುದಿಲ್ಲ. get-caller-identity command Jenkins ಯಾವ AWS identity ಬಳಸಿ ಕೆಲಸ ಮಾಡುತ್ತಿದೆ ಎಂದು ತೋರಿಸುತ್ತದೆ.

5.5 Scenario-based Interview Questions

Q1. Jenkins can run Terraform locally, but the pipeline receives Unable to locate credentials. What do you do?

Answer: Work through these checks:

1. Check whether credentials are configured in Jenkins.
2. Confirm the correct credentials ID is used.
3. Verify that the binding or role-assumption stage is configured correctly.
4. Confirm the pipeline executes in the expected agent and environment.
5. Run aws sts get-caller-identity within the same credential context.

Q2. Should AWS access keys be stored in a Jenkinsfile?

Answer: No. Store credentials in Jenkins Credentials or use a suitable identity mechanism. Avoid committing secrets to Git or printing them in logs.

Q3. Jenkins authentication succeeds, but Terraform receives AccessDenied. Why?

Answer: The authenticated identity may lack the necessary IAM permissions, the permissions boundary or SCP may restrict the action, or the request may target the wrong resource or account.

Q4. Why is an IAM role often preferred over permanent access keys?

Answer: Roles can provide short-lived credentials, simplify rotation, and reduce the risk of long-lived secrets being leaked or misused.

6. Terraform Plan and Apply Automation

6.1 Definition — What are Terraform plan and apply stages?

terraform plan calculates and displays the changes Terraform proposes to make.

terraform apply executes the planned operations to create, update, or destroy infrastructure.

In CI/CD, the plan stage is used for review and the apply stage is used to implement approved changes.

> **Kannada explanation:** plan ಅಂದ್ರೆ AWSನಲ್ಲಿ ಏನೇನು changes ಆಗುತ್ತವೆ ಎಂದು ಮೊದಲು ತೋರಿಸುವುದು. apply ಅಂದ್ರೆ ಆ changes ಅನ್ನು ನಿಜವಾಗಿಯೂ AWSನಲ್ಲಿ ಮಾಡುವುದು.

6.2 Purpose — Why do we use them?

- Preview infrastructure changes before deployment.
- Detect unexpected replacements and deletions.
- Support review and approval workflows.

- Make deployments more controlled.
- Provide a record of proposed and applied changes.

6.3 Working — How do they work?

A basic workflow is:

```
terraform plan -out=tfplan
terraform show tfplan
terraform apply tfplan
```

The saved plan contains the actions Terraform calculated for that run. Applying the saved plan avoids recalculating a different plan at the moment of apply, although Terraform may reject a stale plan if the state or relevant configuration has changed.

Plan files can contain sensitive values. Protect them as artifacts and restrict access.

6.4 Practical Example — Jenkins plan and apply stages

```
stage('Terraform Plan') {
    steps {
        bat 'terraform plan -input=false -out=tfplan'
        archiveArtifacts artifacts: 'tfplan',
                         fingerprint: true
    }
}

stage('Terraform Apply') {
    steps {
        bat 'terraform apply -input=false tfplan'
    }
}
```

This is an illustrative fragment. It does not, by itself, implement manual approval or production security. Those controls should be added before the apply stage.

For production, ensure the plan and apply stages use the same intended configuration, workspace, backend, and relevant input values. The saved plan must be handled securely and applied in the correct context.

> **Kannada explanation:** Plan stage tfplan file save ಮಾಡುತ್ತದೆ. Approval ನಂತರ ಅದೇ plan apply ಮಾಡಬಹುದು. ಆದರೆ plan file sensitive ಆಗಿರಬಹುದು; ಅದಕ್ಕೆ access restrict ಮಾಡಬೇಕು. ಮಧ್ಯದಲ್ಲಿ state ಬದಲಾಗಿದ್ದರೆ plan stale ಆಗಿ apply fail ಆಗಬಹುದು.

6.5 Scenario-based Interview Questions

Q1. Why should you review terraform plan before production apply?

Answer: The plan reveals proposed creations, updates, replacements, and deletions. Reviewing it helps identify unexpected changes before they affect production.

Q2. Jenkins created a plan yesterday. Can you always apply it today?

Answer: No. A saved plan can become stale if state or relevant configuration changes. Terraform may reject it, requiring a fresh plan and review.

Q3. What is the difference between terraform apply and terraform apply tfplan?

Answer: Without a saved plan file, Terraform can calculate a plan and ask for confirmation interactively. With tfplan, Terraform attempts to execute the saved plan; this is useful for controlled CI/CD workflows.

Q4. Should the pipeline automatically apply every Terraform plan?

Answer: Not necessarily. Automatic applies may be acceptable for selected non-production environments with appropriate controls. Production changes generally require stronger review, approval, and permissions.

7. Manual Approval Before Production Deployment

7.1 Definition — What is a manual approval gate?

A manual approval gate is a pipeline stage that pauses execution until an authorized person approves or rejects the next action.

In Terraform CI/CD, approval is often placed after the plan and before the apply stage.

> **Kannada explanation:** Manual approval ಅಂದ್ರೆ Terraform plan ಸಿದ್ಧವಾದ ಮೇಲೆ ಒಬ್ಬ authorized person ಅದನ್ನು ಪರಿಶೀಲಿಸಿ approve ಮಾಡಿದರೆ ಮಾತ್ರ production deployment ಮುಂದುವರಿಯುವುದು.

7.2 Purpose — Why do we use it?

- Prevent accidental production changes.
- Allow engineers to inspect the plan.
- Support change-management processes.
- Separate code development from deployment authorization.
- Reduce the risk of unintended infrastructure destruction.

7.3 Working — How does it work?

1. Jenkins checks out the approved branch.
2. Terraform generates a plan.
3. The pipeline pauses.
4. An authorized approver reviews the plan and change request.
5. If approved, the pipeline proceeds to apply.
6. If rejected or timed out, the deployment does not proceed.

The Jenkins authorization configuration must ensure that only appropriate users can approve production changes.

7.4 Practical Example — Jenkins approval stage

```
stage('Approval') {
    steps {
        timeout(time: 30, unit: 'MINUTES') {
            input message: 'Review the Terraform plan and approve deployment?',
                  ok: 'Approve'
        }
    }
}

stage('Apply') {
    steps {
        bat 'terraform apply -input=false tfplan'
    }
}
```

This is a simplified example. For a secure production pipeline, configure Jenkins permissions and, where appropriate, restrict who may submit the approval. Ensure that the actual saved plan is available and belongs to the intended environment.

> **Kannada explanation:** Plan ನಂತರ pipeline 30 minutesವರೆಗೆ approvalಗಾಗಿ ಕಾಯುತ್ತದೆ. Authorized person approve ಮಾಡಿದರೆ ಮುಂದಿನ stage run ಆಗುತ್ತದೆ. Approval ಸಿಗದಿದ್ದರೆ deployment ಮುಂದುವರಿಯಬಾರದು.

7.5 Scenario-based Interview Questions

Q1. How do you prevent automatic production deployment from a feature branch?

Answer: Restrict production stages to approved branches, use branch protection, enforce pipeline authorization, and require an authorized approval before apply.

Q2. What happens if an approver rejects the deployment?

Answer: The pipeline should stop or follow its explicitly configured rejection handling. The Terraform apply stage must not run.

Q3. Is a Jenkins input step sufficient by itself for production security?

Answer: No. It must be combined with appropriate Jenkins permissions, protected branches, credential isolation, environment controls, and secure handling of plans.

8. Remote State and State Locking in Jenkins

8.1 Definition — What are remote state and state locking?

Remote state stores Terraform state in a remote backend instead of relying only on a local state file.

State locking prevents conflicting Terraform operations from modifying the same state concurrently when the backend supports locking.

For AWS projects, an S3 backend is a common remote-state option. Current Terraform versions support S3-native state locking with use_lockfile = true; older projects may use DynamoDB-based locking, which is deprecated in current Terraform documentation.

> **Kannada explanation:** Terraform stateನಲ್ಲಿ ಯಾವ resources manage ಆಗುತ್ತಿವೆ ಎಂಬ ಮಾಹಿತಿ ಇರುತ್ತದೆ. Jenkins ಮತ್ತು developer ಇಬ್ಬರೂ ಒಂದೇ state ಮೇಲೆ ಒಂದೇ ಸಮಯದಲ್ಲಿ apply ಮಾಡಿದರೆ ಸಮಸ್ಯೆ ಬರಬಹುದು. Remote state ಮತ್ತು locking ಇದನ್ನು ಸುರಕ್ಷಿತವಾಗಿ manage ಮಾಡಲು ಸಹಾಯ ಮಾಡುತ್ತವೆ.

8.2 Purpose — Why do we use them?

- Share state between team members and CI/CD.
- Avoid depending on one developer's local state.
- Reduce the risk of concurrent state modifications.
- Support recovery using backend versioning and backups.
- Improve deployment consistency.

8.3 Working — How does it work?

A simplified S3 backend configuration:

```
terraform {
  backend "s3" {
    bucket       = "company-terraform-state"
    key          = "production/app/terraform.tfstate"
    region       = "us-east-1"
    encrypt      = true
    use_lockfile = true
  }
}
```

Important requirements:

- Create the S3 bucket before initializing this backend.
- Configure the correct AWS credentials and region.
- Enable suitable S3 versioning and access controls.
- Grant the Terraform identity only the necessary permissions.
- Ensure all users and Jenkins jobs use the intended backend and state key.

> **Kannada explanation:** Backend configuration Terraform state ಅನ್ನು ಯಾವ S3 bucket ಮತ್ತು keyನಲ್ಲಿ store ಮಾಡಬೇಕು ಎಂದು ಹೇಳುತ್ತದೆ. use_lockfile = true supported backend locking ಬಳಸುತ್ತದೆ. Bucket, IAM permissions ಮತ್ತು region ಸರಿಯಾಗಿರಬೇಕು.

8.4 Practical Example — Jenkins with an S3 backend

A common project workflow is:

1. Store the backend configuration in the Terraform repository.
2. Configure Jenkins AWS authentication securely.
3. Run terraform init.
4. Generate a plan using the shared state.
5. Review and approve the plan.
6. Apply the approved changes.
7. Check backend versioning and recovery procedures.

Example:

```
stage('Terraform Init') {
    steps {
        bat 'terraform init -input=false'
    }
}

stage('Terraform Plan') {
    steps {
        bat 'terraform plan -input=false -out=tfplan'
    }
}
```

Do not copy a local state file into a production backend without a deliberate migration and backup procedure. Use terraform init -migrate-state when migrating state and carefully follow Terraform's prompts and documentation.

8.5 Scenario-based Interview Questions

Q1. Jenkins and a developer both start Terraform apply against the same backend. What should happen?

Answer: Supported backend locking should allow only one operation to hold the lock at a time. The other operation may wait or fail with a lock error, depending on the backend and operation.

Q2. Jenkins reports a state lock error. Should you immediately run terraform force-unlock?

Answer: No. First verify whether another Terraform process is active. Only unlock a verified stale lock, and follow the team's recovery procedure. Forcing an active lock can risk concurrent state operations.

Q3. Why is local state unsuitable for many team-based CI/CD projects?

Answer: Local state is tied to the working environment and is harder to share and protect consistently. A properly secured remote backend supports collaboration and recovery.

Q4. Does an S3 remote backend automatically protect the state from every deletion?

Answer: No. Use S3 versioning, restrictive IAM permissions, suitable retention or recovery controls, and backups where required.

9. Pipeline Parameters and Environment Management

9.1 Definition — What are pipeline parameters?

Pipeline parameters allow users to provide controlled inputs when starting a Jenkins build. Environment management determines which Terraform configuration, variables, backend, and cloud environment the pipeline targets.

Examples of parameters include:

- ENVIRONMENT: dev, test, or prod
- ACTION: plan or apply
- AWS_REGION: a permitted AWS region

For sensitive production workflows, a separate, tightly controlled pipeline may be safer than allowing arbitrary user-selected actions.

> **Kannada explanation:** Jenkins job start ಮಾಡುವಾಗ environment ಅಥವಾ action ಆಯ್ಕೆ ಮಾಡಲು parameters ಕೊಡಬಹುದು. ಉದಾಹರಣೆಗೆ dev ಅಥವಾ test ಆಯ್ಕೆ ಮಾಡಬಹುದು. ಆದರೆ user ತಪ್ಪಾಗಿ production select ಮಾಡಿದರೂ deployment ಆಗದಂತೆ permissions ಮತ್ತು checks ಇರಬೇಕು.

9.2 Purpose — Why do we use them?

- Reuse a pipeline for multiple environments.
- Avoid duplicating nearly identical Jenkinsfiles.
- Control deployment actions.
- Standardize Terraform variable selection.
- Reduce manual configuration errors.

9.3 Working — How does it work?

Jenkins collects parameter values at build time. The pipeline uses them to select configuration files, working directories, or permitted actions.

Terraform environment values can be provided through *.tfvars files, environment variables, or other controlled input mechanisms.

The selected backend and state must match the intended environment. Merely changing an ENVIRONMENT variable does not automatically isolate Terraform state or AWS accounts.

9.4 Practical Example — Environment parameter

```
pipeline {
    agent any

    parameters {
        choice(
            name: 'ENVIRONMENT',
            choices: ['dev', 'test'],
            description: 'Choose the target environment'
        )
    }

    stages {
        stage('Terraform Plan') {
            steps {
                bat "terraform plan -input=false -var-file=${params.ENVIRONMENT}.tfvars"
            }
        }
    }
}
```

This example intentionally limits the choices to dev and test. It assumes the matching variable files exist and the correct environment-specific backend is already configured.

Production note: Do not accept unrestricted shell commands or arbitrary backend settings from user-controlled parameters. Use an explicit allowlist and enforce AWS account, region, and backend selection in the pipeline.

9.5 Scenario-based Interview Questions

Q1. How do you run the same pipeline for development and testing?

Answer: Use validated parameters or separate environment configurations to select the correct variable file, credentials, and backend.

Q2. A user selects prod accidentally. How can you reduce the risk?

Answer: Use a separate production job or restricted parameter choices, protected branches, separate credentials, environment verification, and mandatory approval.

Q3. Is using -var-file=prod.tfvars enough to guarantee a production deployment is isolated?

Answer: No. State backend, AWS account, region, IAM permissions, and deployment controls must also be correct.

10. Terraform CI/CD Security Best Practices

10.1 Definition — What is Terraform CI/CD security?

Terraform CI/CD security is the set of controls used to protect credentials, state, plans, infrastructure code, pipeline permissions, and production deployments.

> **Kannada explanation:** Terraform pipelineನಲ್ಲಿ secrets leak ಆಗದಂತೆ, ತಪ್ಪು AWS accountನಲ್ಲಿ deployment ಆಗದಂತೆ ಮತ್ತು unauthorized users production changes ಮಾಡದಂತೆ safeguards ಇಡುವುದೇ CI/CD security.

10.2 Purpose — Why do we use it?

- Prevent unauthorized infrastructure changes.
- Protect cloud credentials and sensitive Terraform data.
- Reduce supply-chain and dependency risks.
- Limit the impact of compromised Jenkins jobs.
- Improve auditability and compliance.

10.3 Working — How does it work?

A secure pipeline commonly includes:

- Least-privilege AWS IAM permissions.
- Protected Git branches and reviewed changes.
- Jenkins credentials or temporary role-based authentication.
- Restricted access to plans, logs, workspaces, and artifacts.
- Pinned and reviewed Terraform/provider/module versions.
- Remote state encryption, versioning, and access controls.
- Manual approval for sensitive production changes.
- Separate deployment identities for different environments.
- Regular cleanup of unnecessary build artifacts and secrets.

10.4 Practical Example — Security checklist for Jenkins

Before enabling Terraform deployment, verify:

1. AWS secrets are not stored in Git.
2. The Jenkins credential ID is referenced rather than embedding a secret.
3. The AWS identity has the required minimum permissions.
4. The target account and region are checked.
5. The state backend has suitable access controls and recovery options.
6. Plan output and saved plan files are treated as potentially sensitive.
7. Only authorized users can approve production deployment.

8. Destroy operations have stronger safeguards.
9. Terraform and provider versions are controlled.
10. The pipeline logs do not print secret environment variables.

> **Kannada explanation:** Credentials ಅನ್ನು Gitನಲ್ಲಿ ಇಡಬಾರದು. Production credentials ಎಲ್ಲರಿಗೂ ಸಿಗಬಾರದು. Plan file ಮತ್ತು state fileಗಳಲ್ಲೂ sensitive data ಇರಬಹುದು. ಆದ್ದರಿಂದ ಅವುಗಳಿಗೂ permissions ಮತ್ತು protection ಬೇಕು.

10.5 Scenario-based Interview Questions

Q1. A developer accidentally commits AWS access keys to GitHub. What do you do?

Answer: Take these steps:

1. Revoke or rotate the exposed credentials immediately.
2. Investigate potential unauthorized usage.
3. Review CloudTrail and relevant account activity.
4. Remove the secret from the repository history as appropriate.
5. Add secret scanning and prevention controls.
6. Verify the replacement credentials are stored securely.

Deleting the secret in a later commit alone does not make the exposed credentials safe.

Q2. How do you prevent development Jenkins jobs from modifying production resources?

Answer: Use separate AWS roles or accounts, separate state locations, least-privilege IAM policies, restricted production jobs, and environment-specific approval controls.

Q3. Why should Terraform plan files be protected?

Answer: A saved plan can contain sensitive configuration and values. Restrict access, use controlled artifact retention, and avoid publishing plans to public locations.

11. Terraform Destroy and Approval Controls

11.1 Definition — What is Terraform destroy in CI/CD?

terraform destroy removes infrastructure managed by the selected Terraform configuration and state, subject to dependencies and resource behavior.

In a Jenkins pipeline, a destroy operation should be treated as a high-impact action, particularly when it targets shared or production infrastructure.

> **Kannada explanation:** terraform destroy Terraform manage ಮಾಡುತ್ತಿರುವ resources ಅನ್ನು delete ಮಾಡಲು ಬಳಸಲಾಗುತ್ತದೆ. ತಪ್ಪಾದ environmentನಲ್ಲಿ run ಮಾಡಿದರೆ EC2, VPC ಅಥವಾ EKS ತರಹದ resources delete ಆಗಬಹುದು. ಆದ್ದರಿಂದ ಇದಕ್ಕೆ extra checks ಬೇಕು.

11.2 Purpose — Why do we use it?

- Remove temporary test infrastructure.
- Clean up development environments.
- Reduce unnecessary cloud costs.

- Support controlled infrastructure retirement.
- Execute approved decommissioning workflows.

11.3 Working — How does it work?

A safer destroy workflow is:

1. Verify the Git branch, workspace, backend, AWS account, and region.
2. Generate a destroy plan.
3. Inspect the resources scheduled for destruction.
4. Obtain required approval.
5. Apply the reviewed destroy plan.
6. Verify the outcome and preserve required records.

For production, consider whether resources contain persistent data and whether backups, retention requirements, and dependencies have been addressed.

11.4 Practical Example — Controlled destroy stage

```
stage('Destroy Plan') {
    steps {
        bat 'terraform plan -destroy -input=false -out=destroy.tfplan'
    }
}

stage('Destroy Approval') {
    steps {
        timeout(time: 30, unit: 'MINUTES') {
            input message: 'Confirm the reviewed destroy plan?',
                  ok: 'Approve Destroy'
        }
    }
}

stage('Destroy Apply') {
    steps {
        bat 'terraform apply -input=false destroy.tfplan'
    }
}
```

This is an illustrative fragment. It must be protected by Jenkins authorization, the correct environment selection, and the same careful plan handling described earlier. Do not expose a production destroy action to unrestricted users.

11.5 Scenario-based Interview Questions

Q1. A Jenkins job proposes destroying a production VPC. What do you do?

Answer: Stop the deployment, inspect the destroy plan, verify the target account and state, investigate configuration and dependency changes, and require explicit authorization before any destructive action.

Q2. How would you allow developers to destroy only temporary environments?

Answer: Use dedicated non-production accounts or roles, separate state locations, restricted Jenkins permissions, validated environment selection, and controlled destroy stages.

Q3. Is terraform destroy the same as deleting the Terraform state file?

Answer: No. Destroy attempts to remove managed infrastructure. Deleting the state file removes Terraform's record of the infrastructure and can leave real resources running but unmanaged.

12. Pipeline Failure Troubleshooting

12.1 Definition — What is Terraform pipeline troubleshooting?

Pipeline troubleshooting is the process of identifying and fixing errors that prevent Jenkins from checking out code, authenticating to AWS, initializing Terraform, creating a plan, or applying changes.

> **Kannada explanation:** Jenkins pipeline fail ಆದಾಗ ಯಾವ stageನಲ್ಲಿ error ಬಂದಿದೆ ಎಂದು logs ನೋಡಿ root cause ಕಂಡುಹಿಡಿದು ಸರಿಪಡಿಸುವುದೇ troubleshooting.

12.2 Purpose — Why do we use it?

- Reduce deployment downtime.
- Identify root causes instead of repeatedly retrying.
- Avoid unintended infrastructure changes.
- Improve pipeline reliability.
- Establish repeatable incident-resolution procedures.

12.3 Working — How does it work?

Follow this sequence:

1. Identify the first failed stage.
2. Read the full error message and surrounding logs.
3. Confirm the correct branch, directory, and configuration.
4. Check Terraform and provider versions.
5. Verify credentials, IAM permissions, and target account.
6. Inspect backend configuration and state locking.
7. Fix the underlying issue.
8. Re-run validation and plan before applying.

> **Kannada explanation:** ಕೊನೆಯ error ಮಾತ್ರ ನೋಡದೆ, ಮೊದಲು fail ಆದ stage ಮತ್ತು ಅದರ full logs ಪರಿಶೀಲಿಸಬೇಕು. Error ಸರಿಪಡಿಸಿದ ನಂತರ plan ಮತ್ತೆ run ಮಾಡಿ changes ಸರಿಯಾಗಿವೆ ಎಂದು ಖಚಿತಪಡಿಸಿಕೊಳ್ಳಬೇಕು.

12.4 Practical Example — Common errors and fixes

Error	Possible cause	What to check
terraform: command not found	Terraform not installed or not in PATH	Agent installation and executable path
No configuration files	Wrong working directory	Jenkins workspace and Terraform folder
Unable to locate credentials	AWS credentials not available to the job	Credentials binding, role, or environment
AccessDenied	Insufficient IAM permissions	IAM policies, SCPs, resource policies, account
Backend initialization required	Backend changed or not initialized	Run terraform init with correct settings
Error acquiring the state lock	Another operation or stale lock	Check active Terraform processes and lock owner
No such file: tfplan	Plan file missing or created in another directory	Workspace, stage ordering, artifact handling
Jenkinsfile not found	Wrong branch or script path	Repository, branch, and Jenkinsfile path
Unsupported argument	Provider/schema mismatch or invalid configuration	Provider version, resource documentation, code
Invalid for_each argument	Collection keys or values are not known or valid	Collection expression and known values

12.5 Scenario-based Interview Questions

Q1. Terraform works on your laptop but fails in Jenkins. How do you investigate?

Answer: Compare the Terraform version, provider lock file, working directory, environment variables, AWS identity, backend configuration, network access, and installed dependencies between local and Jenkins environments.

Q2. Jenkins says terraform init must be run again. What do you do?

Answer: Inspect backend and provider changes, then run terraform init in the correct working directory using the intended backend. If state migration is required, back up and follow the appropriate migration process rather than blindly reconfiguring.

Q3. The pipeline fails with a state lock error, but no other build appears active. What next?

Answer: Verify that no Terraform process is still running, inspect the lock details, and follow the backend's recovery procedure. Only remove a confirmed stale lock after ensuring no active operation can be affected.

Q4. Jenkins plan shows many unexpected resource replacements. What do you check?

Answer: Check the selected state, workspace, account, region, variable files, resource identifiers, provider/module versions, and configuration changes. Do not apply until the unexpected replacements are understood.

13. End-to-End AWS Terraform Jenkins Pipeline

13.1 Definition — What is an end-to-end Terraform pipeline?

An end-to-end Terraform pipeline automates the infrastructure lifecycle from source-code checkout through validation, planning, approval, and deployment.

> **Kannada explanation:** GitHubನಲ್ಲಿ code push ಮಾಡಿದಾಗಿನಿಂದ AWSನಲ್ಲಿ resources update ಆಗುವವರೆಗೆ ಬೇಕಾದ ಹಂತಗಳನ್ನು ಒಂದೇ Jenkins pipelineನಲ್ಲಿ automate ಮಾಡುವುದೇ end-to-end pipeline.

13.2 Purpose — Why do we use it?

- Automate repetitive infrastructure operations.
- Standardize deployment steps.
- Integrate code quality checks and change review.
- Make production changes traceable.
- Reduce manual errors.

13.3 Working — How does it work?

A recommended sequence is:

1. Checkout source code.
2. Verify Terraform is available.
3. Initialize the backend and dependencies.
4. Check formatting.
5. Validate the configuration.
6. Authenticate to AWS.
7. Verify the AWS identity and target environment.
8. Create and securely retain the plan.
9. Obtain approval where required.
10. Apply the reviewed plan.
11. Publish a concise build result and clean up sensitive artifacts according to policy.

13.4 Practical Example — Jenkinsfile for a Windows agent

The following is an illustrative starting point for a non-production Terraform workflow. It includes AWS identity verification, validation, plan creation, manual approval, and apply. It assumes a compatible AWS Credentials plugin, a configured backend, an existing Jenkins credential, and Terraform installed on the agent.

```
pipeline {
    agent any

    options {
        timestamps()
        disableConcurrentBuilds()
    }

    parameters {
        choice(
            name: 'ENVIRONMENT',
            choices: ['dev', 'test'],
            description: 'Target environment'
        )
    }

    environment {
        AWS_DEFAULT_REGION = 'us-east-1'
        AWS_REGION         = 'us-east-1'
```

```
        TF_IN_AUTOMATION   = 'true'
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Initialize') {
            steps {
                bat 'terraform version'
                bat 'terraform init -input=false'
            }
        }

        stage('Validate') {
            steps {
                bat 'terraform fmt -check -recursive'
                bat 'terraform validate'
            }
        }

        stage('AWS Identity') {
            steps {
                withCredentials([
                    [
                        $class: 'AmazonWebServicesCredentialsBinding',
                        credentialsId: 'aws-jenkins-credentials'
                    ]
                ]) {
                    bat 'aws sts get-caller-identity'
                }
            }
        }

        stage('Plan') {
            steps {
                withCredentials([
                    [
                        $class: 'AmazonWebServicesCredentialsBinding',
                        credentialsId: 'aws-jenkins-credentials'
                    ]
                ]) {
                    bat "terraform plan -input=false -var-file=${params.ENVIRONMENT}.tfvars -
out=tfplan"
                }
            }
        }

        stage('Approval') {
            steps {
                timeout(time: 30, unit: 'MINUTES') {
                    input message: "Approve Terraform apply for ${params.ENVIRONMENT}?",
                          ok: 'Approve'
                }
            }
        }

        stage('Apply') {
            steps {
                withCredentials([
                    [
                        $class: 'AmazonWebServicesCredentialsBinding',
                        credentialsId: 'aws-jenkins-credentials'
                    ]
```

```
                ]) {
                    bat 'terraform apply -input=false tfplan'
                }
            }
        }
    }

    post {
        always {
            echo 'Terraform pipeline finished. Review the build result and logs.'
        }
    }
}
```

Important production adjustments

- The example assumes the Terraform configuration and environment variable files are in the workspace root. Change the working directory if your project uses a subfolder such as EKS-TF.
- The backend must be configured for the selected environment. A variable file alone does not isolate state.
- Verify the AWS account and region against an explicit allowlist before planning and applying.
- Confirm the AWS credential binding is available and works with the installed plugin version.
- The approval stage should be restricted to authorized approvers in production.
- A saved plan can contain sensitive information. Protect it, avoid public artifact storage, and clean it up under your artifact-retention policy.
- For high-impact environments, use a dedicated production pipeline with separate credentials, state, and stronger deployment controls.
- disableConcurrentBuilds() reduces concurrent executions of this job, but backend locking is still important because other jobs or users can access the same state.

> **Kannada explanation:** ಈ example Windows Jenkins agentಗಾಗಿ ಇದೆ. ನಿಮ್ಮ Terraform files EKS-TF folderನಲ್ಲಿ ಇದ್ದರೆ commands ಆ folderನಲ್ಲಿ run ಆಗುವಂತೆ ಬದಲಾಯಿಸಬೇಕು. Productionಗೆ ಬಳಸುವ ಮೊದಲು credentials, backend, approval permissions ಮತ್ತು account verification ಸರಿಯಾಗಿವೆ ಎಂದು ಖಚಿತಪಡಿಸಿಕೊಳ್ಳಬೇಕು.

13.5 Scenario-based Interview Questions

Q1. Explain your Terraform Jenkins CI/CD project in an interview.

Answer: “I maintain Terraform infrastructure code in GitHub and use Jenkins to automate deployment. The pipeline checks out the configured branch, initializes Terraform, performs formatting and validation checks, authenticates to AWS, generates a plan, and requires approval before applying changes. The project uses controlled credentials and remote state so infrastructure changes are consistent and traceable.”

> **Kannada:** GitHubನಲ್ಲಿ Terraform code ಇಡುತ್ತೇನೆ. Jenkins checkout, init, validate ಮತ್ತು plan run ಮಾಡುತ್ತದೆ. Plan review ಮತ್ತು approval ಆದ ಮೇಲೆ AWSಗೆ apply ಮಾಡುತ್ತೇನೆ. Credentials ಮತ್ತು remote state ಅನ್ನು ಸುರಕ್ಷಿತವಾಗಿ manage ಮಾಡುತ್ತೇನೆ.

Q2. How do you prevent a plan from one environment being applied to another?

Answer: Separate state backends or keys, environment-specific permissions, validated configuration and variable selection, and explicit account checks. Keep the plan and apply steps in the same controlled context.

Q3. Why use disableConcurrentBuilds() and backend locking together?

Answer: The Jenkins option reduces overlap within that job, while backend locking protects the state against concurrent operations from other jobs or users that use the same locking mechanism.

14. Real-Time Production Scenarios and Interview Q&A

Scenario 1 — Jenkins pipeline suddenly cannot access AWS

The pipeline worked yesterday, but today it fails with Unable to locate credentials. What is your approach?

Answer: Work through these steps:

1. Identify the failed stage and inspect its logs.
2. Verify the Jenkins credential ID and credential availability.
3. Confirm the job is running on the expected agent.
4. Check role-assumption or credentials-binding configuration.
5. Run aws sts get-caller-identity in the same credential context.
6. Avoid printing secrets while troubleshooting.

Kannada: ಮೊದಲು credential ID, Jenkins plugin, agent ಮತ್ತು AWS authentication stage ಪರಿಶೀಲಿಸುತ್ತೇನೆ. Secret values ಅನ್ನು logsನಲ್ಲಿ print ಮಾಡುವುದಿಲ್ಲ.

Scenario 2 — Terraform plan proposes destroying an EC2 instance

A plan unexpectedly shows an EC2 instance being destroyed and recreated. What do you do?

Answer: Take these steps:

- Do not approve the plan immediately.
- Check which resource attributes changed.
- Verify the selected workspace, state, account, region, and variable files.
- Inspect changes to for_each keys, count indexes, resource names, and module versions.
- Correct the underlying issue and generate a new plan.

Kannada: Instance destroy ಆಗುವುದಕ್ಕೆ ಯಾವ change ಕಾರಣ ಎಂದು planನಲ್ಲಿ ನೋಡುತ್ತೇನೆ. ಕಾರಣ ಗೊತ್ತಾಗುವವರೆಗೆ apply ಮಾಡುವುದಿಲ್ಲ.

Scenario 3 — State lock error in Jenkins

Jenkins reports that it cannot acquire the state lock. How do you resolve it?

Answer: Work through these steps:

1. Identify whether another build or user is running Terraform.
2. Inspect the lock details and backend.
3. Wait for an active operation to complete.
4. If the lock is stale, follow the backend recovery procedure.
5. Use force-unlock only after confirming that no active operation holds the lock.

Kannada: Lock error ಬಂದ ತಕ್ಷಣ force-unlock ಮಾಡಬಾರದು. ಮೊದಲು ಬೇರೆ Terraform process run ಆಗುತ್ತಿದೆಯೇ ಎಂದು check ಮಾಡಬೇಕು.

Scenario 4 — terraform init works locally but fails in Jenkins

What are the likely causes?

Answer: Differences in Terraform version, provider installation, proxy/network access, credentials, backend permissions, working directory, environment variables, or repository dependencies.

> **Kannada:** Local ಮತ್ತು Jenkins environment ಒಂದೇ ಆಗಿರಬೇಕೆಂದಿಲ್ಲ. Version, PATH, credentials, backend ಮತ್ತು working directory compare ಮಾಡಿ ಸಮಸ್ಯೆ ಪತ್ತೆ ಮಾಡುತ್ತೇನೆ.

Scenario 5 — GitHub push does not trigger Jenkins

How do you troubleshoot it?

Answer: Check the webhook delivery status, repository URL, branch and Jenkinsfile path, job trigger configuration, and network reachability. If Jenkins is accessible only through localhost, configure a secure reachable webhook endpoint or use an alternative trigger mechanism.

Scenario 6 — Production deployment must be stopped after plan generation

The plan has been generated, but the change is no longer approved. What should happen?

Answer: Do not execute apply. Reject or abort the approval step, communicate the outcome, and retain or clean up the plan artifact according to policy. Generate a fresh plan when the change is reviewed again.

Scenario 7 — Terraform state and AWS infrastructure do not match

Jenkins shows unexpected changes because the state and actual infrastructure differ. How do you investigate?

Answer: Confirm the correct backend and state key, inspect the current state and actual AWS resources, investigate manual changes or state migrations, and use a refresh-only plan or other appropriate reconciliation workflow. Do not blindly replace or overwrite state.

Scenario 8 — Terraform provider version differs between developer and Jenkins

How do you maintain consistent behavior?

Answer: Specify compatible Terraform and provider constraints, commit .terraform.lock.hcl, use controlled upgrades, and test provider changes before deployment. Do not upgrade production dependencies without reviewing the plan.

15. Quick Revision — Frequently Asked Interview Questions

Interview question	Key answer
What is Terraform CI/CD?	Automating infrastructure changes through a controlled pipeline.
What is a Jenkinsfile?	A file that defines a Jenkins pipeline.
What is the purpose of terraform init?	Initialize the working directory, backend, and dependencies.
What is the difference between plan and apply?	Plan previews changes; apply executes them.
Why use Jenkins credentials?	To provide controlled access to secrets without hardcoding them.
Why use remote state?	To share and protect state for team-based infrastructure management.
What is state locking?	A mechanism to prevent conflicting operations on the same state.
Why add an approval stage?	To review changes and control high-impact deployments.
What is the purpose of terraform fmt -check?	To check whether Terraform files follow canonical formatting.
What is terraform validate?	A configuration validation command.
How do you prevent accidental destroy operations?	Plan review, restricted permissions, approvals, and separate environment controls.
What do you do when a pipeline fails?	Find the first failed stage, inspect logs, identify the root cause, fix it, and rerun checks.

16. Your Practical Learning Checklist

Part 8 progress: 0/12

- [ ] Explain Terraform and Jenkins CI/CD architecture.
- [ ] Create a Jenkinsfile with checkout, init, fmt, validate, and plan stages.
- [ ] Connect a Jenkins job to a GitHub repository and branch.
- [ ] Configure a secure AWS credentials mechanism.
- [ ] Verify the AWS identity used by Jenkins.
- [ ] Configure and validate an S3 remote backend.
- [ ] Generate a saved Terraform plan.
- [ ] Add an approval stage before apply.
- [ ] Protect production credentials and state.
- [ ] Troubleshoot a Jenkins PATH or working-directory issue.
- [ ] Troubleshoot AWS AccessDenied and credential errors.
- [ ] Investigate unexpected resource destruction safely.

Final interview takeaway

For a Terraform and Jenkins interview, be ready to explain the complete flow:

GitHub → Jenkins Checkout → Terraform Init → Format and Validate → AWS Authentication → Terraform Plan → Approval → Terraform Apply.

The most important production principles are secure credentials, correct state management, environment isolation, plan review, authorized approvals, and systematic troubleshooting.

For practical experience, implement these stages in your existing AWS Terraform project, first using a development environment. Verify the plan and AWS identity before applying changes, and add stronger controls before enabling production deployments.